



Team TROOS

Software Architecture

Specifications

[Introduction](#)

[Architectural patterns](#)

[Architectural Patterns](#)

[Layered Pattern](#)

[Pattern Description](#)

[Layer Structure](#)

[Database Layer](#)

[Governing style: Layered \(N-Tier\) Style](#)

[Applied Tactics](#)

[Quality Attributes Quantified](#)

[Justification](#)

[Tradeoffs](#)

[Pipes & Filters Pattern](#)

[Pattern Description](#)

[Governing Style: Modular Monolith Style](#)

[Architectural Tactics Applied](#)

[Application in ConsultIQ](#)

[CV Parsing Pipeline \(Sequential\)](#)

[Fit Score Computation Pipeline \(Sequential\)](#)

[Quality Attribute Quantified](#)

[Justification](#)

[Tradeoffs](#)

[Modular Monolith Pattern](#)

[Pattern Description](#)

[Governing Style: Modular Monolith Style](#)

[Architectural Tactics Applied](#)

[Internal Module Map](#)

[Quality Attribute Quantified](#)

[Justification](#)

[Tradeoffs](#)

[Modular Monolith Pattern](#)

[Pattern Description](#)

[Governing Style: Modular Monolith Style](#)

[Architectural Tactics Applied](#)

[Internal Module Map](#)

[Internal Events](#)

[Quality Attributes Quantified](#)

[Justification](#)

[Tradeoffs](#)

[Feature-Sliced Design \(FSD\) Frontend Architecture](#)

Pattern Description:

Governing Style: Modular/Domain-Driven Component Style

Architectural Tactics Applied

Quality Attributes Quantified

Justification

Tradeoffs

Consideration

Advantage

Tradeoff

Separation of Concerns

Each feature (e.g., "authentication") is fully isolated, making it easy to remove, refactor, or reuse.

Requires strict architectural discipline and upfront reflection to correctly define the boundaries of each layer, slice, and segment.

Scalability

The hierarchical layers prevent tangled dependencies ("spaghetti code"), making parallel development highly efficient for large teams.

FSD has a learning curve; new team members must understand the strict dependency rules before contributing.

Maintainability

Refactoring is safer and more localized; bugs are isolated to their specific domain.

Can lead to a deeper folder structure, which might feel over-engineered for very small prototypes.

How the Patterns Work Together

Conclusion

Architectural requirements

Design patterns

Strategy pattern

Data Transfer Object (DTO) Pattern

Chain of responsibility

Observer

Orchestrator

Constraints

Scoring Framework

Compliance with POPIA (Protection of Personal Information Act)

3. Usability & Operational Constraints

Quality Requirements to Architectural Decisions

Architectural Diagram

Technology requirements

Frontend & Web Dashboard

Backend API

Database & Caching

Infrastructure & Hosting

CI/CD

Testing

[Deployment requirements](#)

[Deployment Architecture Overview](#)

[Hosting Decisions](#)

[Frontend Hosting: Vercel](#)

[Avenues explored](#)

[Backend Hosting: AWS EC2](#)

[Avenues explored](#)

[Database Hosting: AWS RDS \(PostgreSQL\)](#)

[Containerisation](#)

[Decision to Containerise](#)

[Avenues Explored for Container Runtime](#)

[Multi-Stage Docker Build](#)

[Stage 1 : Builder](#)

[Stage 2 : Production](#)

[Environment Separation: Staging and Production on One Host](#)

[Incident: staging deploys killing production](#)

[CI/CD Pipeline](#)

[Pipeline Overview](#)

[Branching Strategy](#)

[Evolution of the CD Workflow](#)

[Secrets Management](#)

[Security Remediation \(SonarCloud\)](#)

[Hardcoded credentials in docker-compose.yml](#)

[Unpinned GitHub Actions and container hardening](#)

[Networking and HTTPS](#)

[Nginx Reverse Proxy](#)

[SSL/TLS via Certbot](#)

[Known Gaps and Future Work](#)

[CI Pipeline Diagram](#)

[CD Pipeline Diagram](#)

[Deployment diagram](#)

[API Service Contract](#)

[Introduction](#)

[API Design Overview](#)

[API Base Configuration](#)

Introduction

The ConsultIQ Software Architecture Specification (SAS) provides a comprehensive overview of the architectural design, structural decisions, and deployment infrastructure for the system. This document specifies how the system is realized to satisfy those functional, quality, and operational requirements.

Architectural patterns

Architectural Patterns

Layered Pattern

Pattern Description

This serves as the primary architectural pattern for ConsultIQ. It features a clear separation of concerns across the system's layers: presentation, business logic, and data access layer. This decoupling allows the different layers of our system to scale independently and perform consistently under high-volume loads.

Layer Structure

Presentation Layer

This layer serves as the primary gateway for users to interact with the platform. It communicates with the Business Logic Layer via REST API calls and receives no direct database access, increasing security.

Business Layer

The core services for ConsultIQ are found at this tier, which houses all domain rules, algorithms, and process orchestration. This layer operates as a strict mediator between the presentation layer and the data access layer. It propagates requests downward from the user interface and replies upward, applies the necessary business rules, and interfaces with the data layer to persist or retrieve information, ensuring that no database-specific logic leaks into the application's core.

Data Access Layer

This layer manages all database interactions using Prisma ORM. Instead of implementing a custom repository pattern, Prisma Client serves as the direct data access mechanism, which provides type safety abstractions over raw SQL implementation and also allows us to implement features faster. The Business Logic layer interfaces directly with Prisma to persist or retrieve information, ensuring that raw database queries are abstracted away from the application's core logic.

Database Layer

Persistence tier where the database is located. This layer interacts with the data access layer, serving as the long-term storage engine for the system.

Governing style: Layered (N-Tier) Style

This architectural style provides a structural skeleton for the backend and organizes the system horizontally into four layers, assigning each layer a strictly defined responsibility. Communication is strictly sequential; each layer only interacts with the one immediately adjacent to it, with requests flowing down the stack and responses returning up the stack. Ultimately, this structure enforces separation of concerns, enhancing maintainability and security across the entire application.

Applied Tactics

- **Divide the work and rely on clear agreements:** By strictly separating the UI, business rules, and data access, we localize changes and depend on interfaces rather than concrete implementations. This directly addresses the quality attributes of maintainability and testability.
- **Resource Re-use (Connection Pooling & Caching):** To mitigate performance bottlenecks through our data access layer we apply resource-reuse tactics such as TypeORM's built-in connection pool (with a specified maximum pool size) and in-memory caching via the NestJS cache manager (backed by Redis). This directly addresses performance requirements, ensuring the system meets its quantified response-time thresholds under heavy load.

Quality Attributes Quantified

- **Maintainability:** Code modifications and feature enhancements are strictly localized due to contract-based communication between tiers. This architectural boundary ensures a 0% ripple effect into non-adjacent layers when updating internal logic or swapping out underlying technologies, which drastically reduces regression testing overhead and safely allows independent layer updates.
- **Security:** While authentication and business-level authorisation are handled higher in the stack, the Data Access Layer serves as a critical structural safeguard for data persistence. By forcing 100% of database queries through a centralised repository pattern chokepoint, it prevents rogue or accidental direct database access from the

presentation layer and ensures consistent application of POPIA-compliant auditing and data masking.

- **Testability:** Ensures a minimum of 80% unit test coverage per layer. The Business Logic Layer is testable using a mocked database client (ORM), the Presentation Layer with mocked API responses, and the Data Access Layer using an isolated and containerized testing database to ensure queries execute predictably without affecting production data.
- **Performance:** The active use of TypeORM connection pooling and Redis caching prevents network bottlenecks, ensuring database query response times remain under 2 seconds even during peak concurrent loads.

Justification

ConsultIQ serves multiple user roles (Consultants, Project Managers, Consultant Managers, and System Admins). Each role interacts with different parts of the system. A layered architecture provides clear boundaries that map directly onto these roles and allow the system to be developed, tested, and maintained in each layer independently.

Tradeoffs

Consideration	Advantage	Tradeoff
Layer Isolation	A bug in the scoring algorithm is confined to the business logic layer and cannot corrupt the data layer or break the UI independently.	Every new end-to-end feature requires coordinated work across all four layers, increasing the per-feature development effort.
POPIA Structural Compliance	By routing all database interactions through a centralized ORM Client, the system automatically enforces a unified pipeline of data access. This ensures that POPIA-mandated auditing, logging and data masking are applied consistently via middleware.	Intercepting every database query to apply masking and logging adds slight processing overhead. Additionally, database connections must be actively managed (e.g., via connection pooling) to prevent performance bottlenecks under heavy load.
Independent Testability	Each layer can achieve high unit test coverage without the other layers present, enabling fast feedback loops during development.	End-to-end integration tests are still required and more complex to set up, as they must span all four layers simultaneously.

Team Parallelism	Frontend and backend teams can develop simultaneously against agreed REST API contracts.	API contracts must be defined upfront, requiring early design alignment before either side can proceed independently.
-------------------------	--	---

Pipes & Filters Pattern

Pattern Description

This pattern structures the system into a sequence of independent components (filters) connected by a standardised data channel (pipes). Each filter performs a mathematical or structural transformation on the input, and it passes the result to the next filter. Filters are independent, as they do not know what comes before or after them in the pipeline

Governing Style: Modular Monolith Style

This architectural style sets the general rules for structuring and deploying ConsultIQ's domains. Within the Scoring Module, the Pipes & Filters pattern guides internal processing: a strict, sequential chain of stages built inside a single module, not spread across separate services. Since the Scoring Module operates within the same process as the backend, filter-to-filter handoffs are simple function calls instead of network requests, eliminating serialization and latency overhead while maintaining the benefits of modularity and testability. Resource-intensive pipeline runs are delegated to a background job queue, ensuring that processes such as LLM-based CV extraction or multi-factor fit-score recalculations do not block the main application threads.

Architectural Tactics Applied

Specific architectural tactics within this pattern have been engineered to meet our required quality attributes.

- **Abstract Processing Stages:** By encapsulating each stage of a multi-step workflow as an independent, replaceable unit, we prevent the scoring and parsing engines from becoming tangled monoliths. This directly addresses the quality attributes such as modifiability and testability.
- **Defer Processing (Asynchronous Pipeline Execution):** Sequential pipelines are computationally heavy. To guarantee system performance, we apply the tactic of deferring processing. The entire sequential pipeline is triggered asynchronously via

the Event Bus. This ensures that the heavy, multi-step calculations happen on background workers, returning control to the user interface immediately and preserving system responsiveness.

Application in ConsultIQ

CV Parsing Pipeline (Sequential)

When a consultant's CV is uploaded, it passes through a sequential pipeline before the consultant's profile is updated. The pipeline has two entry points, automated (CV upload) and manual (direct data entry), both converging before profile update.

WHEN A CV_{UPLOADED} EVENT IS CONSUMED, THE FOLLOWING FILTER PIPELINE EXECUTES:

Filter	Name	Transformation
F-1	OCR Extraction	Converts the raw PDF or photo to a structured text string.
F-2	LLM Extraction	Sends the text to an LLM; receives structured JSON of skills, certifications, experience, and competency signals.
F-3	Data staging	Maps the JSON payload into a standardised draft schema and pauses pipeline execution. Acts as a gate until the Consultant Manager confirms or corrects the extracted data.
F-4	Profile Update	Writes the validated, corrected data to the Consultant Profile via the Repository; publishes a ProfileUpdated event.

Each filter in the automated path performs one transformation: OCR converts the raw file to text, LLM structures that text, and the final filter stages the data into a draft state. Because each filter depends directly on the output of the previous stage, this automated extraction pipeline executes strictly sequentially. Once the pipeline completes, the system transitions to a manual workflow, presenting the staged data to the Consultant Manager for review before the final event to update the profile is published.

Fit Score Computation Pipeline (Sequential)

When the Scoring Engine evaluates a consultant against a project, the computation is structured as an exhaustive, sequential filter pipeline. A consultant's profile enters the pipe, and each filter computes one scoring dimension, appending its result to the data payload before passing it to the next filter. To ensure complete data visibility for management, the

pipeline does not short-circuit; every active dimension is evaluated even if a previous filter yields a 0% match. Once the payload passes through F-1 to F-5 sequentially, the final filter (F-6) aggregates all appended data into a single composite fit score.

Filter	Scoring Dimension	What It Computes
F-1	Skill Alignment	Percentage match between consultant skills and required project skills.
F-2	Competency Level	Whether consultant proficiency meets minimum project thresholds per skill.
F-3	Geographic Feasibility	Travel cost and time from consultant location to project site via Mapping API.
F-4	Cost vs Billing	Ratio of consultant cost-to-company against project billing budget.
F-5	Availability	Whether consultant is available for the project's start date and duration.
F-6	Weight Aggregation	Applies consultancy-level or project-level weights; produces the final composite fit score.

Filters can be activated or deactivated per project via the two-tier configuration system. A local project might deactivate Filter 3 (geographic) entirely, while a specialised engagement might assign disproportionate weight to Filter 2 (competency).

Quality Attribute Quantified

- **Performance:** By applying the Defer Processing tactic, the exhaustive pipeline is offloaded to background event consumers. This guarantees the UI remains highly responsive, consistently meeting a response time threshold of under 3 seconds even while the backend calculates complex composite scores for a pool of 200+ consultants.
- **Modifiability:** Adding a new scoring dimension requires writing a single new filter and registering it, resulting in zero modifications (0 lines of code changed) to any existing filters or the final aggregation logic.
- **Testability:** Because each filter acts as an isolated processing unit, mock dependencies can be injected, enabling >85% unit test coverage for the scoring and parsing logic in isolation.

Justification

Both the CV Parsing process and the Fit Score computation are inherently sequential, transformational workflows. The Pipes & Filters pattern is a natural architectural fit because it enforces strict single-responsibility principles. By isolating each processing stage, we ensure that individual filters can be rigorously tested, modified, or replaced without affecting adjacent stages or hard-coding complex business logic into a single monolithic function.

Tradeoffs

Consideration	Advantage	Tradeoff
Modularity	Each filter is a self-contained unit, making the pipeline easy to extend and modify	Defining clean filter boundaries requires careful upfront design to avoid filters carrying more load than necessary
Testability	Each filter can be unit tested in complete isolation from the rest of the pipeline	End-to-end pipeline tests are still required to catch integration issues between filters
Error Handling	A failed filter can be isolated and retried without restarting the pipeline	Partial pipeline failures require state management to know which stage failed and resume correctly
Performance	Filters can potentially be parallelised for independent scoring dimensions	Sequential pipelines add latency overhead compared to a single monolithic computation function

Modular Monolith Pattern

Pattern Description

This pattern structures the backend as a single deployable process, internally partitioned into domain-aligned modules (Auth, Consultant, Project, Scoring, Matching, Notification, Analytics), each exposing a narrow, explicit public interface and encapsulating its own internal services, entities, and repositories. Modules communicate exclusively through these

declared interfaces; a module's internals (its repositories, DTOs, or helper services) are never imported directly by another module. Cross-module workflows are handled either as direct, synchronous interface calls or as internally scoped events, chosen per use case based on whether the caller needs the result immediately.

Governing Style: Modular Monolith Style

This architectural style governs how ConsultIQ's domains are deployed and how they communicate. It dictates two things: first, that all domains are compiled and deployed as a single versioned NestJS application rather than as independently deployed services; and second, that domains communicate internally through enforced module boundaries (NestJS module exports and imports) rather than through network calls. This style deliberately trades away independent per-domain scalability and deployment in exchange for eliminating network latency, distributed-transaction complexity, and the infrastructure overhead that a distributed, event-driven system would otherwise require for a system at ConsultIQ's current scale.

Architectural Tactics Applied

- **Restrict Communication Paths (Module Encapsulation):** Each domain module exports a small, explicit provider interface (e.g. ScoringService) via NestJS module metadata; nothing else in the module is importable from outside it. This directly bounds coupling and increases modifiability, since a module's internals can be refactored freely as long as its exported interface's contract is preserved.
- **Defer Processing (In-Process Events + Job Queue):** For workflows that are computationally heavy but do not require an immediate response (CV parsing, bulk fit-score recalculation), modules publish an in-process event via an event emitter and hand the heavy work to a background job queue (BullMQ, backed by Redis). This preserves the responsiveness benefits of asynchronous processing without requiring a separately deployed message broker or service mesh.
- **Single Deployment Unit (Reduce Operational Complexity):** By compiling and deploying the entire backend as one process, we eliminate the need for service discovery, distributed tracing, and per-service CI/CD pipelines that a microservice or fully distributed event-driven system would require.

Internal Module Map

Module	Owns	Exposes
Auth Module	JWT issuance, refresh-token rotation, RBAC guards	AuthService, RolesGuard
Consultant Module	Consultant profiles, CV data, availability	ConsultantService

Project Module	Project definitions, staffing requirements	ProjectService
Scoring Module	Fit-score computation (Pipes & Filters pipeline), scoring configuration	ScoringService
Matching Module	Candidate shortlisting, placement runs	MatchingService
Notification Module	Alerts and gap reports	NotificationService
Analytics Module	Utilisation and placement history	AnalyticsService

Quality Attribute Quantified

- **Performance:** The background worker queues allow the Scoring Engine to concurrently process up to 100 simultaneous ProjectsCreated events without degrading or blocking the HTTP layer.
- **Responsiveness:** By deferring processing, the dashboard returns control to the user immediately, maintaining a UI interaction response time of under 1 second after an event trigger (like a CV upload).
- **Modifiability:** Extensibility is maximised; new services can be added as subscribers with zero modifications to the existing producer codebases.

Justification

The Modular Monolith pattern preserves the decoupling and background-processing benefits the team needs for computationally expensive workflows (LLM-based CV extraction, multi-factor fit-score recalculation) while keeping the system to a single deployable artifact that can be built, tested, and shipped confidently.

Tradeoffs

Consideration	Advantage	Tradeoff
Deployment Simplicity	A single build artifact and a single deployment pipeline drastically reduce operational overhead.	The entire application scales as one unit; a spike in Scoring Module load requires scaling the whole process, not just that module.

Enforced Module Boundaries	Module exports prevent accidental tight coupling between domains, preserving most of the modifiability benefit of microservices.	Careless imports can bypass the intended interface.
In-Process Events	Retains the decoupling and deferred-processing benefits of Pub-Sub (Event Driven) without requiring a separately operated message broker.	All event producers and consumers share the same process and memory space; a fault in one module's event handler can affect the whole application, unlike an isolated service.
Single Point of Failure	There is only one process to monitor, log, and restart, simplifying the operational model.	If the process crashes, the entire backend goes down at once; there is no partial-availability mode where, if one service crashes, the other can still function.
Deployment Simplicity	A single build artifact and a single deployment pipeline drastically reduce operational overhead.	The entire application scales as one unit; a spike in Scoring Module load requires scaling the whole process, not just that module.

Modular Monolith Pattern

Pattern Description

This pattern structures the backend as a single deployable process, internally partitioned into domain-aligned modules (Auth, Consultant, Project, Scoring, Matching, Notification, Analytics), each exposing a narrow, explicit public interface and encapsulating its own internal services, entities, and repositories. Modules communicate exclusively through these declared interfaces; a module's internals (its repositories, DTOs, or helper services) are never imported directly by another module. Cross-module workflows are handled either as direct, synchronous interface calls or as internally scoped events, chosen per use case based on whether the caller needs the result immediately.

Governing Style: Modular Monolith Style

This architectural style governs how ConsultIQ's domains are deployed and how they communicate. It dictates two things: first, that all domains are compiled and deployed as a single versioned NestJS application rather than as independently deployed services; and second, that domains communicate internally through enforced module boundaries (NestJS module exports and imports) rather than through network calls. This style deliberately trades

away independent per-domain scalability and deployment in exchange for eliminating network latency, distributed-transaction complexity, and the infrastructure overhead that a distributed, event-driven system would otherwise require for a system at ConsultIQ's current scale.

Architectural Tactics Applied

- **Restrict Communication Paths (Module Encapsulation):** Each domain module exports a small, explicit provider interface (e.g. ScoringService) via NestJS module metadata; nothing else in the module is importable from outside it. This directly bounds coupling and increases modifiability, since a module's internals can be refactored freely as long as its exported interface's contract is preserved.
- **Defer Processing (In-Process Events + Job Queue):** For workflows that are computationally heavy but do not require an immediate response (CV parsing, bulk fit-score recalculation), modules publish an in-process event via an event emitter and hand the heavy work to a background job queue (BullMQ, backed by Redis). This preserves the responsiveness benefits of asynchronous processing without requiring a separately deployed message broker or service mesh.
- **Single Deployment Unit (Reduce Operational Complexity):** By compiling and deploying the entire backend as one process, we eliminate the need for service discovery, distributed tracing, and per-service CI/CD pipelines that a microservice or fully distributed event-driven system would require.

Internal Module Map

Module	Owns	Exposes
Auth Module	JWT issuance, refresh-token rotation, RBAC guards	AuthService, RolesGuard
Consultant Module	Consultant profiles, CV data, availability	ConsultantService
Project Module	Project definitions, staffing requirements	ProjectService
Scoring Module	Fit-score computation (Pipes & Filters pipeline), scoring configuration	ScoringService
Matching Module	Candidate shortlisting, placement runs	MatchingService
Notification Module	Alerts and gap reports	NotificationService

Analytics Module	Utilisation and placement history	AnalyticsService
-------------------------	-----------------------------------	------------------

Internal Events

The following events are emitted and consumed in-process. Heavy workloads are handed to the BullMQ job queue; lightweight, fast operations are handled synchronously within the same event-handling call.

Event	Emitted By	Handled By
profile.updated	Consultant Module, on CV parsing or manual-entry completion	Scoring Module queues a fit-score recalculation job
project.created	Project Module, on new project definition	Matching Module queues an initial placement run
consultant.assigned	Matching Module, on confirmed placement	Analytics Module updates utilisation and placement history
scoring-config.updated	Admin, on consultancy-level weight change	Scoring Module queues re-evaluation of active projects
skill-gap.detected	Scoring Module, when no adequate match is found	Notification Module alerts the relevant manager with a gap report

Quality Attributes Quantified

- **Performance:** The BullMQ-backed job queue processes up to 100 simultaneous project.created recalculation jobs concurrently via configurable worker concurrency, without blocking the HTTP request thread.
- **Responsiveness:** In-process event emission returns control to the caller immediately (function-call overhead only); heavy recalculation work is deferred to the job queue, maintaining a UI interaction response time of under 1 second after an event trigger such as a CV upload.
- **Modifiability:** New consumers can subscribe to an existing internal event with zero modification to the producer module. Adding a new domain module requires no changes to any existing module's build or deployment configuration, only its own registration in the root application module.

- **Operational Simplicity:** Zero additional deployed services are required to add new domain functionality (no new containers, load balancers, or message brokers), compared to the N additional deployed services a fully distributed, event-driven architecture would require.

Justification

The Modular Monolith pattern preserves the decoupling and background-processing benefits the team needs for computationally expensive workflows (LLM-based CV extraction, multi-factor fit-score recalculation) while keeping the system to a single deployable artifact that can be built, tested, and shipped confidently.

Tradeoffs

Consideration	Advantage	Tradeoff
Deployment Simplicity	A single build artifact and a single deployment pipeline drastically reduce operational overhead.	The entire application scales as one unit; a spike in Scoring Module load requires scaling the whole process, not just that module.
Enforced Module Boundaries	Module exports prevent accidental tight coupling between domains, preserving most of the modifiability benefit of microservices.	Careless imports can bypass the intended interface.
In-Process Events	Retains the decoupling and deferred-processing benefits of Pub-Sub (Event Driven) without requiring a separately operated message broker.	All event producers and consumers share the same process and memory space; a fault in one module's event handler can affect the whole application, unlike an isolated service.
Single Point of Failure	There is only one process to monitor, log, and restart, simplifying the operational model.	If the process crashes, the entire backend goes down at once; there is no partial-availability mode where, if one service crashes, the other can still function.

Feature-Sliced Design (FSD) Frontend Architecture

Pattern Description:

Feature-Sliced Design (FSD) organizes the frontend by using business features rather than technical ones. Technical features dictated that files should be grouped according to what a file is and not what it does. In FSD, the application becomes divided into hierarchical layers: App (global configurations), Pages (full route views), Widgets (complex UI blocks), Features (business logic and interactions), Entities (business data models), and Shared (reusable UI and utilities). The most important rule about FSD is the Dependency Rule (a layer can only import from layers strictly below it). This structure naturally fits modern React environments, of grouping the UI components, API calls, and state management by their domain.

Governing Style: Modular/Domain-Driven Component Style

This style operates by organizing code around the business domains rather than technical types. In practice, we are developing the application around the requirements and not the technical aspects such as the tools that are used. By creating self-contained feature slices, the frontend maintains high cohesion and low coupling. The strict hierarchical dependencies ensure that the business logic remains compartmentalized, preventing spaghetti code.

Architectural Tactics Applied

- **Isolate Domain Logic:** By grouping route-level pages, feature-specific UI pieces, API wrappers, and custom hooks within a dedicated feature folder, we ensure that changes to specific business requirements only affect that localized domain.
- **Enforce strict dependency boundaries:** By adhering to FSD's Layer hierarchy, we prevent circular dependencies and side-to-side imports across features. This structural separation addresses maintainability and system stability.

Quality Attributes Quantified

- **Maintainability & Modifiability:** Because features are strictly isolated, developers can implement changes or roll out new functionality with a significantly reduced risk of breaking other modules. This isolation simplifies bug tracking and accelerates feature development.
- **Usability:** The strict separation allows UI/UX designers to implement design changes with a 0% risk of regressions to the underlying data-fetching or state-management logic.
- **Scalability:** As the project grows, FSD naturally accommodates expansion. The clear layer organization provides a shared architectural vision, allowing multiple developers to work in parallel on different slices without stepping on each other's toes.

Justification

ConsultIQ requires an intuitive UI that users with varying technical expertise (from System Admins to Consultant Managers) can navigate. FSD allows the team to encapsulate complex interactions and API fetching logic within distinct business domains. This ensures the codebase remains predictable and approachable, significantly reducing the cognitive load required to understand or modify complex features like consultant scoring or administrative dashboards.

Tradeoffs

Consideration	Advantage	Tradeoff
Separation of Concerns	Each feature (e.g., "authentication") is fully isolated, making it easy to remove, refactor, or reuse.	Requires strict architectural discipline and upfront reflection to correctly define the boundaries of each layer, slice, and segment.
Scalability	The hierarchical layers prevent tangled dependencies ("spaghetti code"), making parallel development highly efficient for large teams.	FSD has a learning curve; new team members must understand the strict dependency rules before contributing.
Maintainability	Refactoring is safer and more localized; bugs are isolated to their specific domain.	Can lead to a deeper folder structure, which might feel over-engineered for very small prototypes.

How the Patterns Work Together

The architectural patterns and styles chosen for ConsultIQ do not operate in isolation; they are deliberately layered and complementary. The Layered (N-Tier) Style provides the structural skeleton within which all other decisions operate.

Here is how the hierarchy of decisions interacts to form a cohesive system:

- **The Structural Skeleton:** The Layered Pattern builds the backend, enforcing strict downward-only communication. Within its foundation (the Data Access Layer) , the Repository Pattern acts as a strict POPIA compliance chokepoint.
- **The Frontend Interface:** Feature-Sliced Design (FSD) structures the frontend strictly within the N-Tier Presentation Layer. By organizing code around business features rather than technical concerns, it encapsulates all UI, local state, and API interactions within domain-specific slices. These isolated slices communicate exclusively downward with the N-Tier Business Logic Layer.
- **The Internal Processing:** The Pipes & Filters Pattern governs the internal processing logic *within* the Business Logic Layer, turning monolithic scoring tasks into modular, testable stages.
- **The System Communication:** The Event-Driven Style (implemented via the Publish-Subscribe Pattern) governs how these heavy internal components communicate across the system boundary. It allows the strict layers to communicate asynchronously without breaking their structural isolation.

This separation of concerns at the tactical level mirrors the separation of concerns at the structural level, resulting in a system that is consistent from the high-level architecture down to the individual components.

Conclusion

The hybrid architectural approach adopted for ConsultIQ was not chosen based on generic best practices, but because each style, pattern, and tactic addresses a distinct, quantified Quality Requirement.

No single style or pattern was sufficient to satisfy the system's conflicting needs:

- A pure layered style would not elegantly model the sequential nature of CV parsing, nor could it handle score computation synchronously without violating the responsiveness thresholds.
- A pure Pipes & Filters implementation would lack the rigid layered structure required to enforce POPIA data access compliance structurally.
- A pure event-driven system would introduce unnecessary distributed complexity and traceability loss for simple, synchronous CRUD operations.
- Using Feature-Sliced Design (FSD) on the frontend without a Layered backend would leave the system without a unified source of truth or structural discipline.

By carefully combining these styles to govern the system, these patterns to build the components, and these tactics to manage the behaviour, we have consciously accepted operational complexity in exchange for our core goals. The resulting architecture is POPIA-compliant by structure, responsive under heavy computational load, testable at every layer, and safely extensible for future business requirements.

Architectural requirements

Design patterns

Strategy pattern

ConsultIQ serves multiple user roles that must see different versions of the same data. A consultant manager is authorised to see a consultant's cost-to-company rate, but a project manager must never see it. This is both a business rule and a POPIA-adjacent data protection concern.

The intent behind this design pattern is to define a family of algorithms, encapsulates each one, and make them interchangeable at runtime. The caller selects the behaviour by passing a parameter rather than branching internally with hard-coded logic spread across the codebase.

Rather than duplicating mapping logic per role, the role is passed in as a single parameter and one conditional governs the entire data-shaping strategy. As new roles or visibility rules are introduced, this single decision point is the only place that needs updating, keeping the change surface minima

Example:

```
let sidebarItems;

if (currentUserRole === "ADMIN") {
  sidebarItems = adminSidebarItems;
} else if (currentUserRole === "CONSULTANT_MANAGER") {
  sidebarItems = consultantManagerSidebarItems;
} else {
  sidebarItems = consultantSidebarItems;
}
```

Data Transfer Object (DTO) Pattern

ConsultIQ's database schema for a consultant is a normalised relational structure. Exposing this raw shape directly through the API would tightly couple the system's API contract to its database schema. Any schema migration would risk breaking the frontend. It would also expose internal fields (like raw foreign keys or audit columns) that users should never see.

The idea behind the pattern is that it carries data between layers using a plain object that contains no behaviour. It decouples the internal database shape from what is exposed to the outside world, giving each layer a stable, purpose-built contract.

DTOs like **ConsultantListItemDto** and **ConsultantProfileDto** act as a translation layer. The mapping happens once inside the service, flattening nested relations, applying null safety, and renaming fields to match the frontend's expectations. The controller and the frontend never touch raw database objects, meaning a database refactor does not cascade into an API break.

```
const mappedProjects: ProjectListItemDto[] = projects.map((p) => ({
  id: p.id,
  projectName: p.projectName,
  clientName: p.clientName,
  city: p.city,
  province: p.province,
  startDate: p.startDate,
  endDate: p.endDate ?? null,
  teamSize: p.teamSize,
  requiredAllocationPercentage: p.requiredAllocationPercentage,
  clientBillingBudget: Number(p.clientBillingBudget),
  status: p.status,
  skillCount: p.skillCount,
}));
```

Chain of responsibility

Every incoming HTTP request to ConsultIQ must pass several independent checks before reaching the controller: is the user authenticated, do they have the right role, and is the request body shaped correctly? If all of this logic lived inside each controller method, every endpoint would repeat the same boilerplate, and a missed check on a single endpoint would create a security gap.

The goal is to avoid coupling the sender of a request to its receiver. It passes a request along a chain of handlers. Each handler either processes the request and passes it on, or rejects it and stops the chain entirely. No single handler needs to know what comes before or after it.

NestJS guards and pipes form exactly this chain. The AuthGuard checks the JWT token. If it passes, the RoleGuard checks the user's role against the required roles for that endpoint. If that passes, the ValidationPipe checks the request body against the DTO schema. Only if all three pass does the request reach your controller method.

Example of a role guard :

```
@Injectable()

export class RolesGuard implements CanActivate {

  constructor(private readonly reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {

    const isPublic =
this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [

      context.getHandler(),

      context.getClass(),

      .

      . // code //

      .

      const request = context.switchToHttp().getRequest();

      const user = request.user as JwtPayload | undefined;

      if (!user) {
```



```
throw new ForbiddenException(  
  
    'Access denied: no authenticated user found on request. ' +  
  
    'Ensure JwtAuthGuard is applied before RolesGuard.',  
  
);  
  
}
```

Observer

ConsultIQ needs to send real-time alerts and updates to different users whenever an important action takes place. If we hardcode code calls to each user inside the business logic, the system becomes tightly and hard to scale.

The observer pattern solves this by creating a one-to-many relationship. When one object triggers an event, all registered observers are automatically notified. The publisher doesn't need to know the details of each observer.

Observers connect to a **NotificationGateway**. When an event occurs, the gateway loops through its registry and sends updates to all observers.

Orchestrator

ConsultIQ's scoring engine has many steps such as calculating fit scores, applying evaluation rules and saving results. If each service tried to call the next one directly, the logic would get scattered, tangled and hard to maintain.

The orchestrator pattern solves this by putting a central "director" in charge of the workflow. The orchestrator knows the steps and tells each service what to do while the services themselves stay focused only on their specific task.

The scoring orchestrator acts as a coordinator. It fetches project configurations, delegates calculations and collects results. If a new step is added later only the orchestrator sequence is updated.

Constraints

Scoring Framework

- No consultancy-specific business logic, candidate placement rules, scoring factors, or evaluation weights may be hard-coded into the codebase or database schema.

Compliance with POPIA (Protection of Personal Information Act)

- All personal consultant and client data processed, stored, or transmitted by the application must strictly adhere to POPIA requirements.
-

3. Usability & Operational Constraints

- The user interface must accommodate administrative personnel, consultants, and external clients possessing varying levels of technical proficiency.

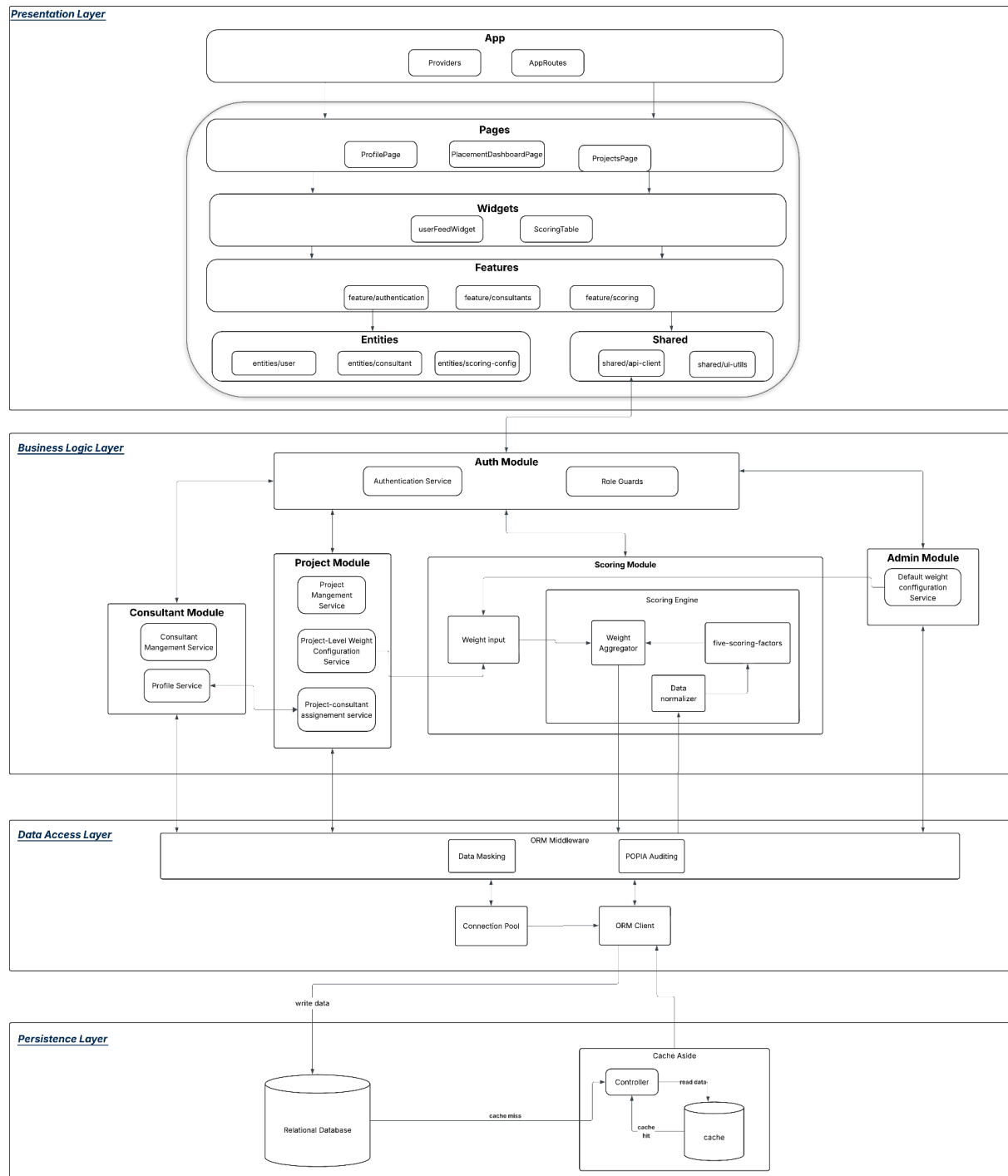
Quality Requirements to Architectural Decisions

Quality Requirement	Architectural Decision
NFR1: Configurability No consultancy-specific business logic may be hardcoded; all placement rules, scoring factors, factor weights, hard exclusion settings, and competency thresholds must be configurable through the admin interface and stored in the database.	Two-tier configuration feature that supports the Scoring Engine with both firm-wide default weights configurable by Admin and project-level overrides configurable by Project Managers. Individual scoring filters (F-1 to F-5) can be activated, deactivated, or reweighted per project without code changes, since filter inclusion and weighting are resolved at execution time rather than compiled into the pipeline.
NFR2: POPIA Compliance Personal consultant data (identity, location, cost-to-company, skills, certifications, CV files) must be stored, access-controlled,	Layered (N-Tier) Style where the Data Access Layer routes 100% of database queries through a centralized ORM client, preventing any direct database access from the Presentation Layer. This is reinforced by Role-Based Access Control (RBAC) via a Chain of Responsibility guard sequence (JwtAuthGuard and RolesGuard), enforcing

consented to, and retained in compliance with POPIA.	authentication and authorization on every request before it reaches the Business Logic Layer.
NFR4: Performance Match run results for up to 200 consultant profiles must return within 5 seconds; all other standard interactions must complete within 3 seconds.	Pipes & Filters pattern for the Fit Score Computation Pipeline, run inside the Modular Monolith's Scoring Module: heavy match runs are deferred via an internal event to a background job worker (BullMQ/Redis), keeping the pipeline's filter-to-filter handoffs as fast in-process function calls rather than network requests. This is reinforced by TypeORM connection pooling and Redis-backed caching in the Data Access Layer, keeping database query response times under 2 seconds under peak load.
NFR5: Security All client-server traffic must be encrypted (TLS 1.2+); sensitive fields (cost-to-company, CV content) must be encrypted at rest; session tokens stored in HTTP-only cookies; every API endpoint must validate the authenticated user's role server-side.	Layered (N-Tier) Style: This architecture enforces strict downward-only communication by implementing two primary defense mechanisms. First, a Chain of Responsibility (JwtAuthGuard, RolesGuard, and ValidationPipe) gates every incoming request to guarantee authentication and data validity before reaching a controller. Second, a strict Data Access Layer chokepoint ensures that the Presentation Layer cannot bypass these data and role restrictions under any circumstances. A strict Data Access Layer chokepoint ensures role validation and data-access restrictions cannot be bypassed regardless of what the Presentation Layer attempts.
NFR8: Maintainability Architectural decisions must be documented; the codebase must conform to agreed coding standards; module boundaries must be enforced so no module imports another module's repository directly.	Modular Monolith Style partitioning the backend into domain modules (Auth, Consultant, Project, Scoring, Matching, Notification, Analytics), each exposing only a narrow, explicit interface (e.g. ScoringService) via NestJS module exports; internals are never imported directly by another module. Combined with the Layered Style's contract-based communication between tiers, this produces a 0% ripple effect into non-adjacent layers when internal logic changes, localising the impact of modifications and keeping regression-testing overhead low.

Table 1: Mapping Quality Requirements to Architectural Decisions

Architectural Diagram



Technology requirements

Frontend & Web Dashboard

React is widely adopted and provides an efficient way to build interactive user interfaces through reusable components. This is particularly useful for developing dynamic dashboards such as the placement recommendation interface, where data updates frequently. TypeScript enhances code reliability by introducing static typing, allowing many errors to be detected during development rather than at runtime. This improves maintainability and reduces debugging effort. To support data visualisation and efficient data handling, libraries such as Recharts will be used to display scoring breakdowns and analytics, while TanStack Query will manage API communication, caching, and synchronization. These tools simplify development and improve performance when working with frequently updated data.

Backend API

NestJS is preferred due to its use of TypeScript, which aligns with the frontend technology stack and reduces the need for developers to switch between languages. It also provides a modern development experience with built-in support for modular design and testing.

Database & Caching

PostgreSQL is chosen for its reliability, strong support for structured data, and advanced features such as row-level security. This is particularly important for supporting multi tenant requirements and ensuring data isolation between different organisations. Redis will be used as both a caching layer and a support system for background job processing. This improves performance and reduces the load on the primary database.

Infrastructure & Hosting

AWS allowing faster development and deployment to run the application in a cloud environment without incurring any costs

CI/CD

Continuous Integration and Deployment will be handled using GitHub Actions, enabling automated testing and deployment processes. Development will be carried out using Visual Studio Code, which offers strong support for TypeScript and a wide range of extensions that improve developer productivity.

Testing

Jest will be used as the primary testing framework for unit and integration testing. It provides a simple and efficient environment for testing business logic, such as the fit score matching engine, ensuring correctness and reliability of core system functionality. Supertest will be used to test API endpoints by simulating HTTP requests and verifying responses. This ensures that the REST API behaves as expected, including correct handling of data, validation, and role-based access control.

React Testing Library will be used for frontend testing, focusing on user interactions and component behaviour rather than implementation details. This approach ensures that the user interface functions correctly from the user's perspective and improves maintainability of tests as the application evolves.

Deployment requirements

Deployment Architecture Overview

At a high level, the system is split across the following components:

- Frontend: React/Vite application hosted on Vercel, served from the domain <https://consult-iq-red.vercel.app>
- Backend: NestJS API, containerised with Docker, running on a single AWS EC2 (Ubuntu) instance, reverse-proxied by Nginx and served over HTTPS at <https://api.consultiq.co.za>.
- Database: PostgreSQL on AWS RDS (af-south-1), accessed exclusively by the backend container.
- Cache/Session store: Redis, running as a container alongside the backend on the same EC2 instance.
- Registry: GitHub Container Registry (GHCR), storing tagged backend images.

- CI/CD: GitHub Actions, running lint, test, SonarCloud analysis, Docker build, and deployment on every relevant push.

Staging and production are both hosted on the same EC2 instance as separate, isolated Docker Compose stacks, differentiated by container naming, port mapping, and environment files. This decision is expanded on in the CI/CD section.

Hosting Decisions

Frontend Hosting: Vercel

The frontend remained on Vercel throughout the project rather than being migrated into the same containerised AWS stack as the backend.

Avenues explored

Option	Notes	Verdict
Keep frontend on Vercel	Already deployed and working; automatic preview deployments per PR; zero server management.	Selected
Migrate frontend to AWS (S3 + CloudFront, or containerised on the same EC2 instance)	Considered mid-project to consolidate everything under one cloud provider and demonstrate broader AWS proficiency.	Rejected

The migration to AWS was evaluated seriously enough that the decision was deliberately paused and reconsidered before committing engineering time to it. The conclusion was that moving the frontend would introduce meaningful risk (DNS cutover, cache invalidation strategy, build pipeline rewrite) for very little benefit, since Vercel already satisfied every actual requirement - a publicly reachable, HTTPS-served, automatically deployed frontend. The decision was made to keep the frontend on Vercel and focus all containerisation effort on the backend, where it was actually required by the project brief.

Backend Hosting: AWS EC2

The backend is hosted on a single AWS EC2 instance running Ubuntu, in the af-south-1 (Cape Town) region, matching the RDS database region.

Avenues explored

Option	Notes	Verdict
--------	-------	---------

AWS EC2 (self-managed)	Full control over the box; supports Docker Compose directly; team already had an instance provisioned and working.	Selected
AWS ECS / Fargate	Proper container orchestration, auto-scaling, no host management. Considered too complex to configure correctly within the remaining project timeline.	Rejected
AWS App Runner	Simplest AWS-native container service; minimal configuration. Rejected for offering less control over networking (Nginx/Certbot) and being comparatively more expensive for a low-traffic capstone workload.	Rejected

EC2 with Docker Compose was chosen because it satisfied the project's containerisation requirement, is the pattern most recognisable to assessors and the wider industry, and was achievable within the remaining time before Demo 2 without introducing orchestration complexity the team did not yet need.

Database Hosting: AWS RDS (PostgreSQL)

The database runs on a managed PostgreSQL instance on AWS RDS rather than as a container alongside the backend.

Running Postgres as a container on the same EC2 instance was briefly considered for simplicity, but was rejected: a managed RDS instance removes the operational burden of backups, patching, and storage management from the team, and keeps the database's failure domain separate from the backend container's - a backend container crash or redeploy cannot take the database down with it. The RDS endpoint (`consultiq-db.cdscm66a89wq.af-south-1.rds.amazonaws.com`) is only reachable from the backend; it is not exposed publicly.

Containerisation

Decision to Containerise

The backend originally ran directly on the EC2 instance under PM2 (a Node.js process manager), without containerisation. This was the fastest path to a working deployment early in the project and is documented in the CI/CD Pipeline section as the pipeline's first working version. Containerisation was a deliberate, later decision, not the starting point.

The motivation for moving to Docker was threefold: the project brief explicitly required containerisation as a deliverable; a containerised build guarantees the same artifact runs in

staging, production, and on any developer's machine, removing an entire class of "works on my machine" bugs; and containers give a clean boundary for the environment-separation and secrets-management work described later in this document.

Avenues Explored for Container Runtime

Option	Notes	Verdict
PM2 direct on EC2 (status quo)	Simple, already working, no new tooling to learn. Did not satisfy the project's containerisation requirement and left staging/production environments unisolated from each other.	Rejected
Docker + Docker Compose on EC2	Satisfies the containerisation requirement; supports multiple isolated stacks (staging/production) on one host via Compose project naming; achievable within roughly one to two focused days.	Selected
AWS ECS / Fargate	Would have been the natural next step after Docker Compose, but was considered too large a scope change this late in the project.	Rejected for this phase; noted as future work

The estimated effort for the Docker migration was scoped up front - roughly 30-45 minutes each for the backend and frontend Dockerfiles, a similar amount of time for the local and production Compose files, an hour for the GitHub Actions build-and-push step, one to two hours for migrating the EC2 host off PM2, and one to two hours for HTTPS via Nginx - for a total of roughly one to two days of focused work. The riskiest parts were correctly anticipated to be environment variables behaving differently inside containers, inter-container networking (the backend being unable to reach Redis or the database), and Prisma migrations needing to run at the correct point in the container lifecycle.

Multi-Stage Docker Build

The backend Dockerfile uses a two-stage build so that the final production image is as small and clean as possible.

Stage 1 : Builder

- Installs all dependencies, including dev dependencies, using npm ci.
- Runs npx prisma generate to generate the Prisma client.
- Runs npm run build to compile the NestJS application from TypeScript to JavaScript.

Stage 2 : Production

- Starts again from a clean base image.
- Installs only production dependencies using `npm ci --omit=dev`, keeping dev tooling and test libraries out of the runtime image.
- Copies over only the compiled dist folder and the generated Prisma client from the builder stage.
- Runs as a non-root user.
- On container start, runs `npx prisma migrate deploy` to apply any pending migrations, then starts the app with `node dist/src/main.js`.

Dependency installation always happens inside the Docker build itself, via `npm ci` in the Dockerfile, never as a separate step on the CI runner beforehand. This was a deliberate check made against the actual pipeline mid-project: the concern was that installing dependencies on the runner before the Docker build would mean the built image could silently diverge from what CI had tested. The pipeline was confirmed to already do this correctly - `npm ci` runs twice, once with dev dependencies in the builder stage (needed to compile the app) and once without in the production stage (nothing in that tooling is needed to run the compiled output, and excluding it keeps the image smaller and reduces attack surface).

This gives reproducibility (the same lockfile-driven install runs on every build, regardless of what is cached on the CI runner or the EC2 host), eliminates drift (the image is fully self-contained rather than depending on whatever happens to be in `node_modules` on the host), and takes advantage of Docker layer caching (because `package*.json` is copied and `npm ci` is run before the rest of the source is copied in, unrelated code changes do not force a full dependency reinstall).

Environment Separation: Staging and Production on One Host

Both staging and production run as separate Docker Compose stacks on the same EC2 instance, each with its own backend and Redis service, its own port mapping, and its own environment file (`.env.staging` and `.env.production`, both stored only on the EC2 host itself, never in the repository).

Incident: staging deploys killing production

A production incident occurred where deploying to staging caused the production stack to go down. Investigation traced the cause to `docker-compose.staging.yml` and `docker-compose.prod.yml` both using identical service keys (redis, backend) and neither file specifying an explicit Compose project name. Docker Compose derives a project name from the directory by default when none is set, so both stacks were being treated as the same project. The GitHub Actions deploy workflow ran `docker-compose -f docker-compose.staging.yml down` on every push to the dev branch, and because both stacks shared the same project label, this down command matched and stopped the production containers as well as the staging ones.

The fix was to add an explicit name: `consultiq-staging` and `name: consultiq-prod` to the respective Compose files, which scopes Compose's container matching to each stack independently. The deprecated top-level version: `'3.8'` key was also removed from both files as part of the same change, since Compose itself already flags it as obsolete. A one-time manual recreation of both stacks was required on the server so that already-running containers picked up the corrected project labels; from that point forward, staging and production deploys are fully isolated from one another.

CI/CD Pipeline

Pipeline Overview

Two logically separate pipelines exist, both implemented as GitHub Actions workflows: a continuous integration (CI) pipeline that runs on every pull request, and a continuous deployment (CD) pipeline that runs on every push to `dev` (staging) or `main` (production).

The full sequence, in order:

- 1. Trigger: a commit is pushed, or a pull request is opened, against a relevant branch.
- 2. Lint and test: ESLint runs, then the Jest test suite runs, then SonarCloud analyses the code for quality and security issues. All of this happens before any Docker image is built, so broken or low-quality code is never packaged.
- 3. Build the Docker image: the multi-stage build described above runs, producing a production-ready image.
- 4. Tag the image: the image is tagged with both a branch-derived tag (staging or latest) and the Git commit SHA, so every deployed image can be traced back to the exact commit that produced it.
- 5. Push to registry: the tagged image is pushed to GitHub Container Registry (GHCR).
- 6. Deploy: depending on the branch, GitHub Actions SSHes into the EC2 instance and brings up the corresponding Compose stack with the newly pushed image.

Branching Strategy

The branching model maps directly onto the two pipelines:

```
feature/branch  -> push          -> nothing runs
      |
      | open PR into dev -> CI pipeline runs (lint, type check, tests, SonarCloud)
      |
      | PR approved and merged into dev
      |
      | dev branch -> CD pipeline deploys to STAGING
      |
      | release branch cut before each demo, tested, merged into main
```

|
main branch -> CD pipeline deploys to PRODUCTION

Requiring the CI status check to pass before a pull request can be merged is enforced at the repository settings level, so untested or failing code cannot reach dev.

Evolution of the CD Workflow

The deployment workflow went through several distinct versions rather than being correct on the first attempt. The earliest working version deployed directly to a PM2 process over SSH:

steps:

```
- uses: actions/checkout@v4
- uses: appleboy/ssh-action@v1.0.3
  with:
    host: ${ secrets.EC2_HOST }
    username: ${ secrets.EC2_USERNAME }
    key: ${ secrets.EC2_PRIVATE_KEY }
    script: |
      cd /home/ubuntu/consultiq
      git pull origin main
      cd backend
      npm ci
      npm run build
      pm2 restart consultiq-api --update-env || pm2 start dist/main.js --name consultiq-api
      pm2 save
```

This version is documented here because several of the issues that had to be debugged to get it working recurred, in modified form, once the pipeline moved to Docker - the underlying causes (secrets, host state, memory limits) were the same class of problem in both cases.

Secrets Management

Secrets are handled at two distinct levels, deliberately kept separate:

- GitHub Actions secrets (EC2_HOST, EC2_USERNAME, EC2_PRIVATE_KEY, PROD_API_URL, STAGING_API_URL, and registry credentials) are stored in the repository's Actions secrets and are only ever referenced via `${ secrets.NAME }` in workflow files, never printed or embedded in build output.
- Runtime application secrets (database credentials, JWT secret, third-party API keys) live only on the EC2 host itself, as `.env.staging` and `.env.production`, and are never committed to the repository or baked into the Docker image. Each Compose stack references its own file via `env_file`.

This split means a compromised CI log can never leak a running application secret, and a compromised application secret does not expose the deployment credentials needed to reach the host.

Security Remediation (SonarCloud)

SonarCloud is run as a mandatory step in the CI pipeline, gating code quality and security before a Docker image is ever built. Two categories of finding were remediated over the course of the project.

Hardcoded credentials in docker-compose.yml

The Compose file initially scored a C security rating, driven by a hardcoded database password embedded directly in the POSTGRES_PASSWORD field, a literal JWT_SECRET value, and a DATABASE_URL connection string that embedded the same password a second time — counted as a double violation by the scanner. SonarCloud pattern-matches on credential-shaped fields (PASSWORD:, SECRET:, connection strings) regardless of whether the surrounding context is local development or production.

The fix was to extract every secret value into a root-level .env file, excluded from git via .gitignore, and replace every literal value in the Compose file with \${VAR} substitution syntax. A companion .env.example with placeholder values was committed so teammates could set up their own local environment without needing the real secrets. Git history was also checked to confirm no real credential had been committed in an earlier commit, since a scanner flag can persist even after the current file is fixed if a prior commit still contains the value.

Unpinned GitHub Actions and container hardening

A second remediation pass addressed GitHub Actions referenced by a mutable tag (e.g. @v4) rather than a pinned commit SHA — a supply-chain risk, since a tag can be moved to point at different, potentially compromised code after the fact. Actions in the workflow files were pinned to full SHA hashes.

The frontend's Nginx container was also flagged for running as root inside the container. This was fixed by configuring the Nginx image to run as a non-root user and moving its listening port from 80 to 8080, since ports below 1024 require root privileges to bind.

Networking and HTTPS

Nginx Reverse Proxy

Nginx runs on the EC2 host (outside the Docker network) and reverse-proxies external requests on ports 80/443 to the backend container listening internally on port 3000. This keeps the container itself unaware of TLS termination and lets certificate renewal be managed entirely at the host level.

```
server {  
    listen 80;  
    server_name api.consultiq.co.za;  
  
    location / {  
        proxy_pass http://localhost:3000;  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;
```

```
proxy_set_header Connection 'upgrade';
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_cache_bypass $http_upgrade;
}
}
```

SSL/TLS via Certbot

HTTPS was provisioned using Certbot with the Nginx plugin, requesting a Let's Encrypt certificate for `api.consultiq.co.za`. Certbot configured Nginx automatically and registered a scheduled renewal task, so the certificate renews itself before expiry without manual intervention. Once issued, all downstream configuration was updated to point at the HTTPS endpoint rather than the previous plain-HTTP/IP-based address: the frontend's `VITE_API_URL` environment variable on Vercel, the `PROD_API_URL` and `STAGING_API_URL` GitHub Actions secrets, and the backend's CORS allow-list in `main.ts`.

Known Gaps and Future Work

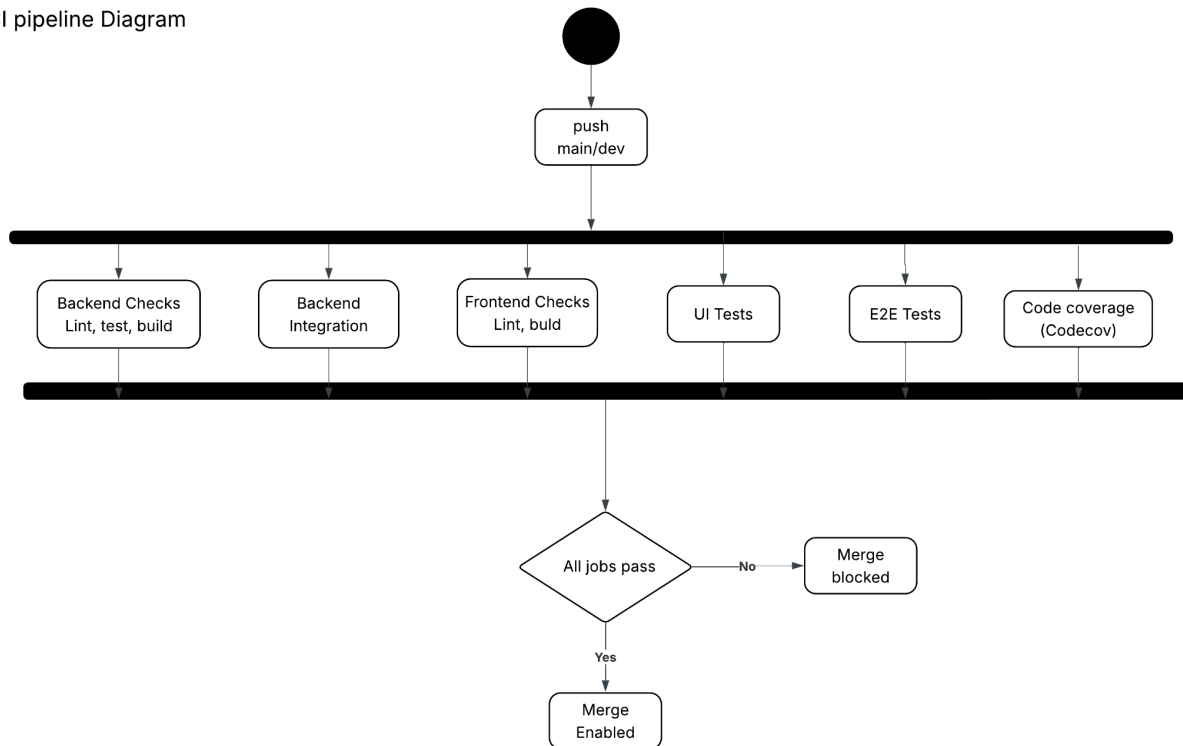
A deployment audit was carried out against the project's stated infrastructure requirements — a live, publicly reachable system; parity between staging and production; containerisation; and secrets kept out of source control. All four are satisfied by the current setup. The audit also surfaced gaps that are not required by the project brief but are worth recording as deliberate, known limitations rather than oversights:

- No automatic rollback strategy - a failed deploy is not automatically rolled back to the previous SHA-tagged image; a rollback today would be a manual redeploy of an older image tag.

None of these blocked the project's requirements, but each is a natural next step if ConsultIQ continued past the capstone timeline - most straightforwardly, adding a health-check curl step after the deploy and keeping the previous image tag available for a one-command rollback.

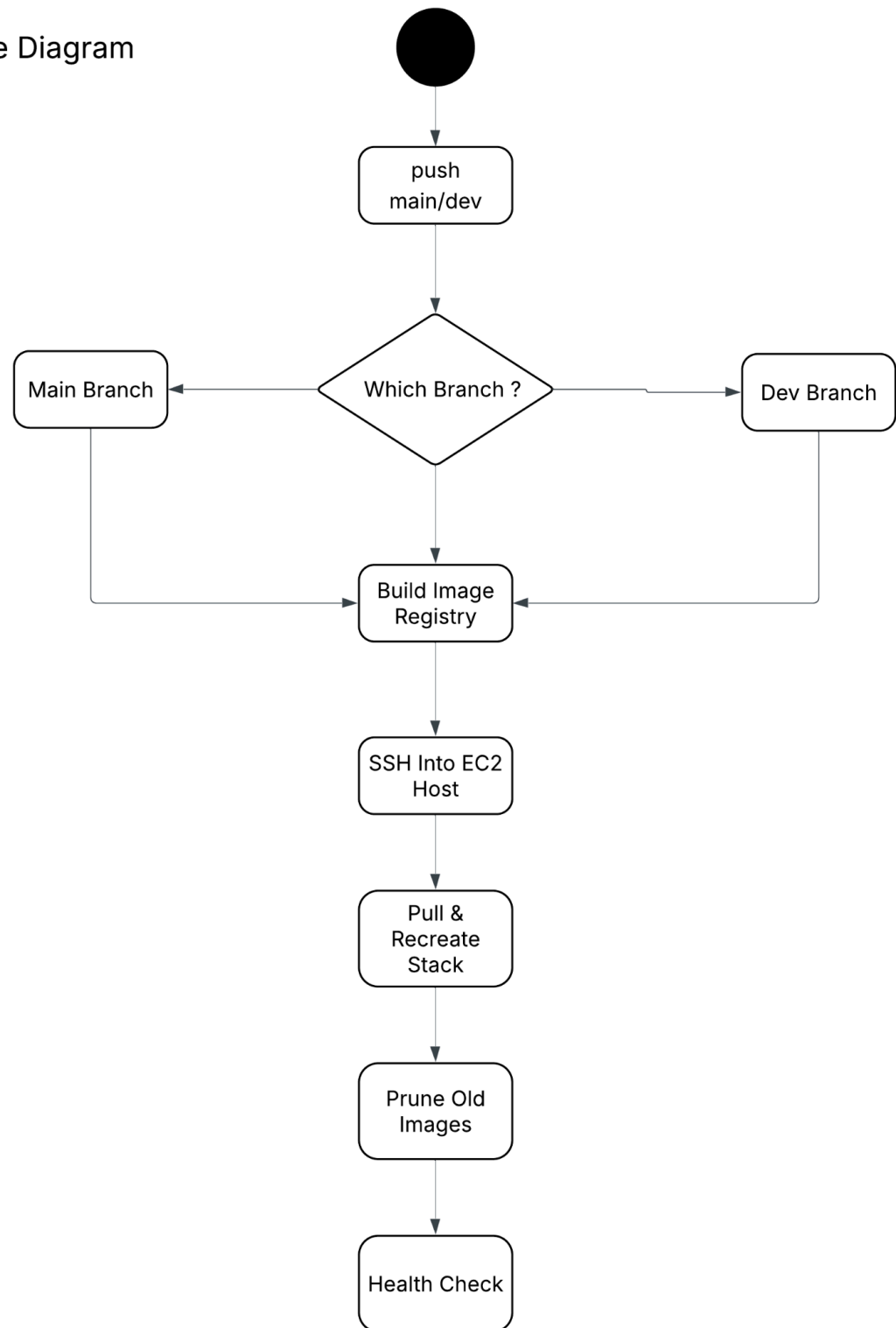
CI Pipeline Diagram

CI pipeline Diagram

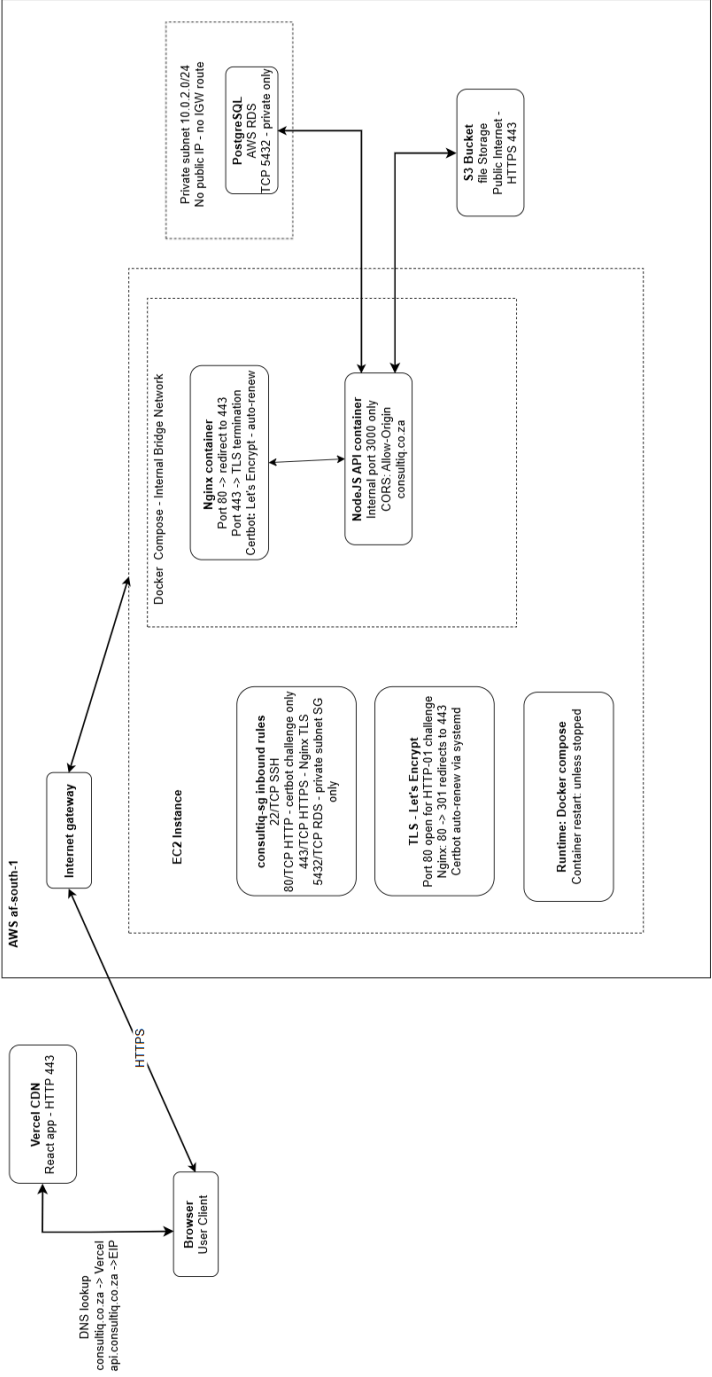


CD Pipeline Diagram

CD pipeline Diagram



Deployment diagram



API Service Contract

Introduction

ConsultIQ is a data-driven platform designed to assist consultancy firms in optimally matching consultants to project requirements. The system centralises consultants' profiles and project specifications and enables intelligent matching through structured data and configurable logic.

This document defines the API Service Contract for demo 1, covering the following functional requirements:

- R1: Authentication and Authorisation
- R2: Consultant Profile Management
- R3: Project Specification Management
- R4: Scoring Configuration Management
- R5: Match Run Execution
- R6: Notification Management
- R7: Administration

The API enables secure communication between the frontend and the backend while enforcing role-based access control and ensuring data integrity.

API Design Overview

The ConsultIQ API follows RESTful design principles, ensuring clarity, scalability, and maintainability.

Design Decisions

- Architecture: REST API using HTTP methods (GET, POST, PUT, PATCH)
- Authentication: JSON Web Tokens (JWT)
- Authorisation: Role-Based Access Control (RBAC)
- Data Format: JSON for all requests and responses
- Statelessness: Each request contains all required authentication data
- Error Handling: We use standardised error responses

across all endpoints

Security Considerations

- User authentication is handled using JWT tokens issued upon login

- JWT tokens have a fixed expiration time set for 8 hours post-login
- Passwords are securely hashed
- Sensitive data, such as the cost to the company, are restricted at API level
- All protected endpoints require a valid bearer token

API Base Configuration

Base URL

<https://api.consultiq.co.za/v1>

Authentication Header

Authorization: Bearer <JWT_TOKEN>

Standard Error Format

```
{
  "error": "ERROR_CODE",
  "message": "Human-readable message"
}
```

API Endpoints

Authentication (R1)

User Registration

Endpoint: POST /auth/register

Description: Creates a new pending user account on behalf of an authorised system user. Upon successful registration, a verification email is sent to the user to complete account setup, including username/password creation and POPIA consent acceptance.

Request body:

```
{
  "fullName": "Jane Smith",
  "email": "jane.smith@consultiq.com",
  "role": "CONSULTANT"
}
```

Success Response (201 Created):

```
{
  "userId": "uuid",
  "email": "jane.smith@consultiq.com",
  "role": "CONSULTANT",
  "status": "PENDING_VERIFICATION",
  "message": "Verification email sent successfully"
}
```

Error Response (409 Conflict):

```
{
  "error": "EMAIL_ALREADY_EXISTS",
  "message": "A user with this email already exists"
}
```

Error Response (400 Bad Request):

```
{
  "error": "VALIDATION_ERROR",
  "message": "Email address is invalid"
}
```

Complete Registration

Endpoint: POST /auth/complete-registration

Description: Completes the onboarding process for a registered user by validating the verification token, setting the username and password, and capturing POPIA consent acceptance. Upon successful completion, the user account status is updated from PENDING VERIFICATION to ACTIVE.

Request body:

```
{
  "token": "verification-token",
  "username": "janesmith",
  "password": "Password123!",
  "acceptedPopia": true
}
```

Success Response (200 OK):

```
{
  "userId": "uuid",
  "username": "janesmith",
  "status": "ACTIVE",
  "message": "Account registration completed successfully"
}
```

Error Response (400 Bad Request):

```
{
  "error": "INVALID_TOKEN",
  "message": "The verification token is invalid or has expired"
}
```

Error Response (400 Bad Request):

```
{
  "error": "INVALID_POPIA_CONSENT",
  "message": "POPIA consent must be accepted to activate the account"
}
```

Retrieve POPIA Content

Endpoint: GET /popia

Description: Retrieves the POPIA consent page content, including the

platform's privacy, data processing, and consent terms that users must review and accept before account activation.

Success Response (200 OK):

```
{
  "title": "POPIA Consent Agreement",
  "content": "By using ConsultIQ, you consent to the processing
and storage of your personal information in accordance
with the Protection of Personal Information Act
(POPIA)."
```

Success Response (200 OK):

```
{
  "error": "POPIA_CONTENT_UNAVAILABLE",
  "message": "Unable to retrieve POPIA content at this time"
```

User Login

Endpoint: POST /auth/login

Description: Authenticates a user with email and password. Returns a JWT on success. Does not specify whether email or password is wrong on failure (R1.1.1, R1.1.3).

Request body:

```
{
  "email": "jane.smith@consultiq.com",
  "password": "Password123!"
}
```

Success Response (200 OK): Login successful. Returns JWT and role-appropriate dashboard redirect hint.

```
{
  "token": "jwt-token",
  "expiresIn": 28800,
  "user": {
    "id": "uuid",
    "fullName": "John Doe",
    "role": "Project Manager"
  }
}
```

Error Response (401 Unauthorised): Invalid credentials. The message does not specify whether the email or password was incorrect.

```
{
  "error": "INVALID_CREDENTIALS",
  "message": "The email address or password is incorrect."
}
```

User Logout

Endpoint: POST /auth/logout

Description: Invalidates the current user's session token immediately (R1.2.1, R1.2.2). Success Response (200 OK):

```
{
  "message": "Logout successful"
}
```

Error Response (401 Unauthorised): User is not authenticated.

```
{
  "error": "UNAUTHORISED",
  "message": "Authentication is required"
}
```

Consultant Profile Management (R2)

List All Consultants

Endpoint: GET /consultants

Access Control: Consultant Manager only

Description: Retrieves a paginated list of consultant profiles. Supports search and filtering. Success Response (200 OK): Paginated list of consultants

```
{
  "page": 1,
  "total": 2,
  "consultants": [
    {
      "id": "uuid",
      "fullName": "Jane Smith",
      "city": "Johannesburg",
      "availabilityStatus": "Available",
      "primarySkills": ["Java", "Spring Boot"]
    }
  ]
}
```

Error Response (401 Unauthorised): Authentication token missing or expired.

```
{
  "error": "UNAUTHORISED",
  "message": "Authentication is required. Please log in."
}
```

Error Response (403 Forbidden): User does not have the required role permission for this action (R1.3.6).

```
{
  "error": "FORBIDDEN",
  "message": "You do not have permission to perform this action."
}
```

Notes:

- Search is case-insensitive (R2.4.1)
- Filters are applied using AND logic (R2.4.2)
- Cost-to-company is not included in list view

Create Consultant Profile

Endpoint: POST /consultants/create

Access Control: Consultant Manager only

Description: Creates a new consultant profile, including skills and certifications. All data is saved in a single transaction.

Request Body:

```
{
  "fullName": "Jane Smith",
  "email": "jane@consultiq.com",
  "city": "Johannesburg",
  "province": "Gauteng",
  "costToCompanyRate": 650,
  "availabilityStatus": "Available",
  "skills": [
    {
      "skillName": "Java",
      "yearsOfExperience": 4,
      "confidenceLevel": 8
    }
  ],
  "certifications": [
    {
      "certificationName": "AWS Certified Cloud Practitioner",
      "issuingBody": "AWS"
    }
  ]
}
```

Success Response (201 Created): Consultant profile created successfully.

```
{
  "message": "Consultant created successfully",
  "consultantId": "uuid"
}
```

Error Response (400 Bad Request): Validation error

```
{
  "error": "VALIDATION_ERROR",
  "message": "Duplicate skills are not allowed."
}
```

Error Response (401 Unauthorised): Authentication token missing or expired.

```
{
  "error": "UNAUTHORISED",
}
```

```
  "message": "Authentication is required. Please log in."
}
```

Error Response (403 Forbidden): User does not have the required role permission for this action.

```
{
  "error": "FORBIDDEN",
  "message": "You do not have permission to perform this
action." }
```

Notes:

- Duplicate skills are not allowed (R2.1.6)
- Skills and certifications are saved atomically (R2.1.7)

Get Consultant Profile

Endpoint: GET /consultants/{consultantId}

Access Control: Consultant Manager and/or Admin

Description: Retrieves full details of a consultant profile.

Success Response (200 OK):

```
{
  "id": "uuid",
  "fullName": "Jane Smith",
  "email": "jane@consultiq.com",
  "city": "Johannesburg",
  "province": "Gauteng",
  "costToCompanyRate": 650,
  "availabilityStatus": "Available",
  "skills": [
    {
      "skillName": "Java",
      "yearsOfExperience": 4,
      "confidenceLevel": 8,
      "competencyLevel": "Intermediate"
    }
  ],
  "certifications": []
}
```

Error Response (401 Unauthorised): Authentication token missing or expired.

```
{
  "error": "UNAUTHORISED",
  "message": "Authentication is required. Please log in."
}
```

Error Response (403 Forbidden): User does not have the required role permission for this action.

```
{
  "error": "FORBIDDEN",
  "message": "You do not have permission to perform this
```



```
action." }
```

Error Response (404 Not Found): The requested resource does not exist.

```
{
  "error": "NOT_FOUND",
  "message": "The requested resource could not be found."
}
```

Update Consultant Profile

Endpoint: PUT /consultants/{consultantId}

Access Control: Consultant Managers and Consultants (changes made by consultants need approval)

Description: Updates an existing consultant profile. The incoming skill list replaces the existing skills entirely in a single transaction.

Request Body:

```
{
  "fullName": "Jane Smith",
  "email": "jane@consultiq.com",
  "city": "Pretoria",
  "province": "Gauteng",
  "costToCompanyRate": 700,
  "availabilityStatus": "Unavailable",
  "skills": [
    {
      "skillName": "React",
      "yearsOfExperience": 3,
      "confidenceLevel": 7
    }
  ],
  "certifications": []
}
```

Success Response (200 OK):

```
{
  "message": "Consultant updated successfully",
  "consultantId": "uuid"
}
```

Error Response (400 Bad Request):

```
{
  "error": "VALIDATION_ERROR",
  "message": "Invalid input data"
}
```

Notes:

- Skill set is replaced atomically (R2.2.3)
- Competency levels are recalculated on update (R2.2.2)

Preview Derived Competency Level

Endpoint: POST /consultants/competency-preview

Description: Returns a derived competency level based on years of experience and confidence level without saving any data.

Request Body:

```
{
  "yearsOfExperience": 4,
  "confidenceLevel": 8
}
```

Success Response (200 OK): Derived competency level.

```
{
  "competencyLevel": "Beginner"
}
```

Error Response (400 Bad Request):

```
{
  "error": "VALIDATION_ERROR",
  "message": "Invalid input values"
}
```

Notes:

- Used for real-time UI feedback (R2.1.4)
- No data is persisted

Remove Consultant

Endpoint: DELETE /consultants/{consultantId}

Access Control: Consultant Manager and Admin

Description: Removes a consultant profile from the system. This action may only be performed by an authorised consultant manager or admin user. The associated consultant account and related profile mappings are removed or deactivated according to business rules.

Success Response (200 OK):

```
{
  "message": "Consultant removed successfully"
}
```

Error Response (404 Not Found)

Project Specification Management (R3)

List All Projects

Endpoint: GET /projects

Access Control: Project Manager only

Description: Retrieves a paginated list of project specifications.

Success Response (200 OK): List of project specifications.

```
{
  "page": 1,
  "total": 2,
  "projects": [
    {
      "id": "uuid",
      "projectName": "Banking System Upgrade",
      "clientName": "ABC Bank",
      "city": "Johannesburg",
      "province": "Gauteng",
      "status": "Open",
      "startDate": "2026-05-01",
      "requiredSkillCount": 3
    }
  ]
}
```

Notes:

- The list view shows project summary information only

Create Project Specification

Endpoint: POST /projects

Access Control: Project Manager only

Description: Creates a new project specification with required skill requirements. The project is automatically assigned an initial status of Open.

Request Body:

```
{
  "projectName": "Banking System Upgrade",
  "clientName": "ABC Bank",
  "city": "Johannesburg",
  "province": "Gauteng",
  "startDate": "2026-05-01",
  "endDate": "2026-08-30",
  "teamSize": 4,
  "requiredAllocationPercentage": 100,
  "clientBillingBudget": 1200,
  "skillRequirements": [
    {
      "skillName": "Java",
      "minimumCompetencyLevel": "Intermediate",
      "mandatory": true
    }
  ]
}
```

Success Response (200 OK): Project specification created with status Open.

```
{
```

```
    "message": "Project created successfully",
    "projectId": "uuid",
    "status": "Open"
}
```

Error Response (400 Bad Request):

```
{
  "error": "VALIDATION_ERROR",
  "message": "Duplicate skill requirements are not allowed."
}
```

Error Response (403 Forbidden):

```
{
  "error": "FORBIDDEN",
  "message": "You do not have permission to perform this
action." }
```

Get Project Specification

Endpoint: GET /projects/{projectId}

Access Control: Project Manager only

Description: Retrieves the full details of a project specification, including all required skills and mandatory flags.

Success Response (200 OK):

```
{
  "id": "uuid",
  "projectName": "Banking System Upgrade",
  "clientName": "ABC Bank",
  "city": "Johannesburg",
  "province": "Gauteng",
  "startDate": "2026-05-01",
  "endDate": "2026-08-30",
  "teamSize": 4,
  "requiredAllocationPercentage": 100,
  "clientBillingBudget": 1200,
  "status": "Open",
  "skillRequirements": [
    {
      "skillName": "Java",
      "minimumCompetencyLevel": "Intermediate",
      "mandatory": true
    }
  ]
}
```

Error Response (404 Not Found):

```
{
  "error": "NOT_FOUND",
  "message": "Project specification not found"
}
```

Update Project Specification

Endpoint: PUT /projects/{projectId}

Access Control: Project Manager only

Description: Updates an existing project specification, including project details and skill requirements.

Request Body:

```
{
  "projectName": "Banking System Upgrade",
  "clientName": "ABC Bank",
  "city": "Pretoria",
  "province": "Gauteng",
  "startDate": "2026-05-01",
  "endDate": "2026-09-15",
  "teamSize": 5,
  "requiredAllocationPercentage": 100,
  "clientBillingBudget": 1300,
  "skillRequirements": [
    {
      "skillName": "Java",
      "minimumCompetencyLevel": "Expert",
      "mandatory": true
    },
    {
      "skillName": "React",
      "minimumCompetencyLevel": "Intermediate",
      "mandatory": false
    }
  ]
}
```

Success Response (200 OK):

```
{
  "message": "Project updated successfully",
  "projectId": "uuid"
}
```

Error Response (400 Bad Request)

```
{
  "error": "VALIDATION_ERROR",
  "message": "Invalid project input data"
}
```

Error Response (403 Forbidden)

```
{
  "error": "FORBIDDEN",
  "message": "You do not have permission to perform this action"
}
```

Update Project Status

Endpoint: PATCH /projects/{projectId}/status

Access Control: Project Manager only

Description: Updates the status of a project specification. Request Body:

```
{
  "status": "In Progress"
}
```

Success Response (200 OK)

```
{
  "message": "Project status updated successfully",
  "projectId": "uuid",
  "status": "In Progress"
}
```

Error Response (400 Bad Request):

```
{
  "error": "INVALID_STATUS",
  "message": "Invalid project status"
}
```

Error Response (403 Forbidden)

```
{
  "error": "FORBIDDEN",
  "message": "You do not have permission to perform this action"
}
```

Scoring Configuration Management (R4)

Get Scoring Configuration

Endpoint: GET /scoring-config

Access Control: Admin only

Description: Retrieves the firm-wide scoring configuration used by the matching engine. If no configuration exists, the system seeds and returns the default set of scoring factors (Skill Alignment, Competency Level, Availability, Location, Cost to Company).

Success Response (200 OK):

```
[
  {
    "factorName": "SKILL_ALIGNMENT",
    "weight": 40,
    "active": true,
    "hardExclusionEnabled": false
  },
  {
    "factorName": "COMPETENCY_LEVEL",
    "weight": 30,
    "active": true,

```

```
    "hardExclusionEnabled": false
  },
  {
    "factorName": "AVAILABILITY",
    "weight": 15,
    "active": true,
    "hardExclusionEnabled": false
  },
  {
    "factorName": "LOCATION",
    "weight": 10,
    "active": true,
    "hardExclusionEnabled": false
  },
  {
    "factorName": "COST_TO_COMPANY",
    "weight": 5,
    "active": true,
    "hardExclusionEnabled": false
  }
]
```

Error Response (401 Unauthorised):

```
{
  "error": "UNAUTHORISED",
  "message": "Authentication is required. Please log in."
}
```

Update Scoring Configuration

Endpoint: PUT /scoring-config

Access Control: Admin only

Description: Updates the firm-wide scoring configuration. Each active scoring factor is upserted in a single transaction, and the previous and new values are written to a scoring configuration audit log.

Request Body:

```
{
  "scoringFactors": [
    {
      "factorName": "SKILL_ALIGNMENT",
      "weight": 45,
      "active": true,
      "hardExclusionEnabled": false
    },
    {
      "factorName": "COMPETENCY_LEVEL",
      "weight": 25,
      "active": true,
      "hardExclusionEnabled": false
    }
  ]
}
```

```

    },
    {
      "factorName": "AVAILABILITY",
      "weight": 15,
      "active": true,
      "hardExclusionEnabled": false
    },
    {
      "factorName": "LOCATION",
      "weight": 10,
      "active": true,
      "hardExclusionEnabled": false
    },
    {
      "factorName": "COST_TO_COMPANY",
      "weight": 5,
      "active": true,
      "hardExclusionEnabled": false
    }
  ]
}

```

Success Response (200 OK): Returns the updated set of scoring factors.

```

[
  {
    "factorName": "SKILL_ALIGNMENT",
    "weight": 45,
    "active": true,
    "hardExclusionEnabled": false
  }
]

```

Error Response (400 Bad Request): Active factor weights do not sum to 100.

```

{
  "error": "VALIDATION_ERROR",
  "message": "Active factors weights must sum to 100. Current sum: 90" }

```

Error Response (400 Bad Request): No active scoring factors were supplied.

```

{
  "error": "VALIDATION_ERROR",
  "message": "At least one scoring factor must be active."
}

```

Notes:

- Active factor weights must sum to exactly 100
- Every update is written to a scoring configuration audit log, including the

admin user, previous values and new values

Get Project Scoring Override

Endpoint: GET /projects/{projectId}/scoring-override

Access Control: Assigned Project Manager only

Description: Retrieves the project-specific scoring factor overrides for the given project. Success Response (200 OK):

```
[
  {
    "factorName": "SKILL_ALIGNMENT",
    "overrideWeight": 50,
    "active": true
  },
  {
    "factorName": "LOCATION",
    "overrideWeight": 50,
    "active": true
  }
]
```

Error Response (403 Forbidden):

```
{
  "error": "FORBIDDEN",
  "message": "Only the assigned Project Manager can manage scoring overrides for this project."
}
```

Update Project Scoring Override

Endpoint: PUT /projects/{projectId}/scoring-override

Access Control: Assigned Project Manager only

Description: Creates or updates project-specific scoring factor overrides. Each supplied factor is upserted in a single transaction against the project.

Request Body:

```
{
  "factors": [
    {
      "factorName": "SKILL_ALIGNMENT",
      "overrideWeight": 50,
      "active": true,
      "hardExclusionEnabled": false
    },
    {
      "factorName": "LOCATION",

      "overrideWeight": 50,
      "active": true,
      "hardExclusionEnabled": false
    }
  ]
}
```

```
]
}
```

Success Response (200 OK): Returns the updated set of project scoring overrides.

```
[
  {
    "factorName": "SKILL_ALIGNMENT",
    "overrideWeight": 50,
    "active": true
  },
  {
    "factorName": "LOCATION",
    "overrideWeight": 50,
    "active": true
  }
]
```

Error Response (400 Bad Request): Active override weights do not sum to 100.

```
{
  "error": "VALIDATION_ERROR",
  "message": "Active factor weights must sum to exactly 100. Current
sum: 80." }
```

Error Response (403 Forbidden):

```
{
  "error": "FORBIDDEN",
  "message": "Only the assigned Project Manager can manage scoring overrides for this
project." }
```

Notes:

- Active override weights must sum to exactly 100
- Where no active override exists for a project, the firm-wide scoring configuration is used

Delete Project Scoring Override

Endpoint: DELETE /projects/{projectId}/scoring-override

Access Control: Assigned Project Manager only

Description: Removes all project-specific scoring factor overrides for the given project, reverting the project to the firm-wide scoring configuration. Deletion must be explicitly confirmed.

Request Body:

```
{

  "confirm": true
}
```

Success Response (200 OK):

```
{
```

```
    "message": "Project scoring override removed successfully"
  }
}
```

Error Response (400 Bad Request): Deletion was not confirmed.

```
{
  "error": "VALIDATION_ERROR",
  "message": "Deletion must be explicitly confirmed by setting confirm to true."
}
```

Error Response (403 Forbidden):

```
{
  "error": "FORBIDDEN",
  "message": "Only the assigned Project Manager can manage scoring overrides for this project."
}
```

Match Run Execution (R5)

Execute Match Run

Endpoint: POST /projects/{id}/match-run

Access Control: Authenticated user (assigned Project Manager)

Description: Executes a new match run for the specified project. Active consultants are scored against the project's required skills using the resolved scoring weights (project override, falling back to firm-wide configuration). The run and its results are persisted, and a ranked list of scored consultants is returned.

Success Response (201 Created):

```
{
  "runId": "uuid",
  "results": [
    {
      "consultantId": "uuid",
      "consultantName": "Jane Smith",
      "consultantEmail": "jane@consultiq.com",
      "finalScore": 87.5,
      "rank": 1,
      "factorBreakdown": [
        {
          "factorName": "SKILL_ALIGNMENT",
          "weight": 40,
          "rawScore": 90,
          "weightedScore": 36
        }
      ]
    },
    {
      "isPlaced": false
    }
  ]
}
```

Error Response (404 Not Found): Project does not exist.

```
{
  "error": "NOT_FOUND",
  "message": "Project: {projectId} is not found"
}
```

Error Response (400 Bad Request): Project has no required skills.

```
{
  "error": "VALIDATION_ERROR",
  "message": "Cannot execute match run: Project has no required skills."
}
```

Error Response (400 Bad Request): Project status does not allow a match run.

```
{
  "error": "VALIDATION_ERROR",
  "message": "Match run can only be initialized for open and in-progress projects. Project Sta}
}
```

Error Response (400 Bad Request): No eligible consultants exist.

```
{
  "error": "VALIDATION_ERROR",
  "message": "No consultants to match against an active project"
}
```

Notes:

- Only active consultants are considered for scoring
- Consultants with an overlapping active placement are flagged as isPlaced
- Consultants that fail scoring are excluded from the run and counted separately
- The scoring weight configuration used is captured as a snapshot on the match run record

Get Match Run

Endpoint: GET /projects/{id}/match-run/{runId}

Description: Retrieves the ranked, scored consultant results for a previously executed match run. Success Response (200 OK):

```
[
  {
    "consultantId": "uuid",

    "consultantName": "Jane Smith",
    "consultantEmail": "jane@consultiq.com",
    "finalScore": 87.5,
    "rank": 1,
    "factorBreakdown": [
      {
        "factorName": "SKILL_ALIGNMENT",
```

```

        "weight": 40,
        "rawScore": 90,
        "weightedScore": 36
      }
    ],
    "isPlaced": false
  }
]

```

Error Response (404 Not Found):

```

{
  "error": "NOT_FOUND",
  "message": "Match run {runId} not found for project {projectId}" }

```

Get Match Run Statistics

Endpoint: GET /projects/{id}/match-run/{runId}/stats

Description: Retrieves summary statistics for a previously executed match run, including how many consultants were evaluated, excluded, matched, and already placed.

Success Response (200 OK):

```

{
  "totalEvaluated": 12,
  "totalExcluded": 2,
  "totalMatched": 10,
  "totalPlaced": 3
}

```

Error Response (404 Not Found):

```

{
  "error": "NOT_FOUND",
  "message": "Match run {runId} not found"
}

```

Notification Management (R6)

Get Notifications

Endpoint: GET /notifications

Description: Retrieves the authenticated user's most recent, non-archived notifications, ordered from newest to oldest, capped at 50 records.

Success Response (200 OK):

```

[
  {
    "id": "uuid",
    "userId": "uuid",

```

```
    "title": "New match run completed",
    "body": "Your match run for Banking System Upgrade has
finished.", "link": "/projects/uuid/match-run/uuid",
    "isRead": false,
    "isArchived": false,
    "createdAt": "2026-07-30T09:15:00.000Z"
  }
]
```

Error Response (401 Unauthorised):

```
{
  "error": "UNAUTHORISED",
  "message": "Authentication is required. Please log in."
}
```

Get Archived Notifications

Endpoint: GET /notifications/archived

Description: Retrieves the authenticated user's archived notifications, ordered from newest to old est, capped at 50 records.

Success Response (200 OK):

```
[
  {
    "id": "uuid",
    "userId": "uuid",
    "title": "Consultant profile updated",
    "body": "Jane Smith's profile was updated and requires
review.", "link": "/consultants/uuid",
    "isRead": true,
    "isArchived": true,
    "archivedAt": "2026-07-28T14:02:00.000Z",
    "createdAt": "2026-07-27T11:40:00.000Z"
  }
]
```

Error Response (401 Unauthorised):

```
{
  "error": "UNAUTHORISED",

  "message": "Authentication is required. Please log in."
}
```

Mark Notification As Read

Endpoint: PATCH /notifications/{id}/read

Description: Marks a single notification belonging to the authenticated user as read. Success Response (200 OK):

```
{
  "id": "uuid",
  "isRead": true
}
```

Error Response (404 Not Found):

```
{
  "error": "NOT_FOUND",
  "message": "The requested resource could not be found."
}
```

Mark All Notifications As Read

Endpoint: PATCH /notifications/read-all

Description: Marks all of the authenticated user's notifications as read. Success Response (200 OK):

```
{
  "message": "All notifications marked as read",
  "updatedCount": 6
}
```

Error Response (401 Unauthorised):

```
{
  "error": "UNAUTHORISED",
  "message": "Authentication is required. Please log in."
}
```

Archive Notification

Endpoint: PATCH /notifications/{id}/archive

Description: Archives a single notification, removing it from the default notifications list. Success Response (200 OK):

```
{
  "id": "uuid",
  "isArchived": true,
  "archivedAt": "2026-07-31T08:00:00.000Z"
}
```

```
}
```

Error Response (404 Not Found):

```
{
  "error": "NOT_FOUND",
  "message": "The requested resource could not be found."
}
```

Administration (R7)

List All Users

Endpoint: GET /admin/users

Access Control: Admin only

Description: Retrieves a paginated list of non-admin users, excluding users still pending verification, along with aggregate counts of active and suspended users.

Success Response (200 OK):

```
{
  "data": [
    {
      "id": "uuid",
      "fullName": "Jane Smith",
      "email": "jane@consultiq.com",
      "role": "CONSULTANT",
      "status": "ACTIVE",
      "createdAt": "2026-06-01T09:00:00.000Z"
    }
  ],
  "meta": {
    "totalRecords": 42,
    "currentPage": 1,
    "totalPages": 5,
    "activeUsers": 38,
    "suspendedUsers": 4
  }
}
```

Error Response (403 Forbidden):

```
{
  "error": "FORBIDDEN",
  "message": "You do not have permission to perform this action."
}
```

Activate User

Endpoint: PATCH /admin/users/{userId}/activate

Access Control: Admin only

Description: Sets the status of the target user account to ACTIVE.

Success Response (200 OK):

```
{
  "message": "User activated successfully"
}
```

Error Response (404 Not Found):

```
{
  "error": "NOT_FOUND",
}
```



```
  "message": "User does not exist"
}
```

Suspend User

Endpoint: PATCH /admin/users/{userId}/suspend

Access Control: Admin only

Description: Sets the status of the target user account to SUSPENDED.

Success Response (200 OK):

```
{
  "message": "User suspended successfully"
}
```

Error Response (404 Not Found):

```
{
  "error": "NOT_FOUND",
  "message": "User does not exist"
}
```

Assign User Role

Endpoint: PUT /admin/users/{userId}/role

Access Control: Super Admin and Admin only

Description: Changes the role of a target user. An admin user cannot change their own role, cannot assign the SUPER ADMIN role, and cannot change the role of an existing SUPER ADMIN account. Every role change is written to an admin audit log.

Request Body:

```
{
  "role": "CONSULTANT_MANAGER"
}
```

Success Response (200 OK):

```
{
  "message": "Role of user {userId} changed from CONSULTANT to
CONSULTANT_MANAGER successfully"
}
```

Error Response (400 Bad Request): Admin attempted to change their own role.

```
{
  "error": "VALIDATION_ERROR",
  "message": "You cannot change your own role."
}
```

Error Response (400 Bad Request): Attempted to assign the SUPER ADMIN role.

```
{
```

```
"error": "VALIDATION_ERROR",
"message": "The SUPER_ADMIN role cannot be assigned through
this method." }
```

Error Response (404 Not Found):

```
{
  "error": "NOT_FOUND",
  "message": "Target user with id {userId} not found."
}
```

List All Projects (Admin)

Endpoint: GET /admin/projects

Access Control: Admin only

Description: Retrieves a paginated list of non-archived project specifications for administrative oversight, along with the total record count and the absolute total including archived projects.

Success Response (200 OK):

```
{
  "data": [
    {
      "id": "uuid",
      "projectName": "Banking System Upgrade",
      "status": "Open",
      "clientName": "ABC Bank",
      "createdAt": "2026-05-01T09:00:00.000Z",
      "budget": 1200
    }
  ],
  "meta": {
    "totalRecords": 18,
    "absoluteTotalRecords": 21,
    "currentPage": 1,
    "totalPages": 2
  }
}
```

```
}
```

Error Response (403 Forbidden):

```
{
  "error": "FORBIDDEN",
  "message": "You do not have permission to perform this
action." }
```

Archive Project (Admin)

Endpoint: PATCH /admin/projects/{projectId}/archive

Access Control: Admin only

Description: Archives a project specification and records the action, together with the performing admin, in the project audit log.

Success Response (200 OK):

```
{
  "message": "Project archived successfully"
}
```

Error Response (404 Not Found):

```
{
  "error": "NOT_FOUND",
  "message": "Project does not exist"
}
```

Error Response (401 Unauthorised): Admin identity could not be resolved from the request.

```
{
  "error": "UNAUTHORISED",
  "message": "User ID not found in request"
}
```

Unarchive Project (Admin)

Endpoint: PATCH /admin/projects/{projectId}/unarchive

Access Control: Admin only

Description: Restores a previously archived project specification to Open status and records the action in the project audit log.

Success Response (200 OK):

```
{
  "message": "Project unarchived successfully"
}
```

Error Response (404 Not Found):

```
{
  "error": "NOT_FOUND",
  "message": "Project does not exist"
}
```

Error Response (401 Unauthorised): Admin identity could not be resolved from the request.

```
{
  "error": "UNAUTHORISED",
  "message": "User ID not found in request"
}
```